Topic	What It Is	Why It Matters
Tries & Suffix Trees	Tree-like data structures for storing strings efficiently.	Used in autocomplete, spell checkers, DNA sequencing, and text editors.
Disjoint Set (Union-Find)	Structure to track elements partitioned into disjoint sets.	Core for Kruskal's MST, cycle detection, and network connectivity problems.
Segment Trees	Binary tree for range queries (sum, min, max).	Efficient for dynamic range queries in competitive programming.
Fenwick Tree (BIT)	Space-efficient alternative to Segment Tree.	Great for prefix sums, cumulative frequencies, and range queries.
Heavy-Light Decomposition	Splits a tree into chains for fast path queries.	Optimizes many tree query problems in contests.
Tarjan's Algorithm	DFS-based algorithm for finding strongly connected components.	Key in graph theory, compilers, and circuit analysis.
KMP Algorithm	String matching with preprocessing of the pattern.	Faster pattern searching than brute force.
Rabin-Karp Algorithm	Hashing-based string matching.	Ideal for multi-pattern search and plagiarism detection.
Bitmask Dynamic Programming	Represents subsets with bitmasks inside DP states.	Solves problems like TSP and subset optimization efficiently.
Dynamic Programming on Trees	Applies DP transitions over tree structures.	Solves hierarchical optimization problems on graphs/trees.
Johnson's Algorithm	All-pairs shortest paths for sparse weighted graphs.	Often faster than Floyd–Warshall on sparse graphs.
Bellman–Ford Edge Cases	Shortest paths with possible negative edges.	Detects negative cycles; useful in finance/arbitrage modeling.
Persistent Data Structures	Preserve past versions after modifications.	Enables undo/versioning; common in functional programming.
Advanced Graph Matching	Algorithms like Hopcroft–Karp for maximum bipartite matching.	Crucial for scheduling, assignments, and resource allocation.